南開大學
Nankai University

计算机学院

并行程序设计实验报告

Lab1 CPU 架构编程

Cache 与超标量对程序性能的影响

姓名：柯云超

学号：2413575

专业：计算机科学与技术

学院：计算机学院

2026 年 3 月 26 日

摘要

本文在真实 Ubuntu x86_64 机器上完成 Lab1 CPU 架构编程实验，围绕矩阵列内积与浮点数组求和两个问题，分析 cache 局部性与超标量执行对程序性能的影响。实验分别实现了矩阵的逐列访问、行优先访问、行优先加 4 路展开，以及求和的链式累加、两路独立累加、四路独立累加和两两归约等版本；测试过程中采用固定公式生成数据、单核绑核、warm-up 与累计时长阈值计时，以减小短程序测量抖动。结果表明：矩阵实验中，修正访问顺序是决定性优化，在 $n = 2048$ 时行优先加 4 路展开相对平凡算法达到 31.62 倍加速；求和实验中，四路独立累加最高达到 3.88 倍加速，而两两归约在较大规模下因中间结果读写开销反而慢于链式累加。结合反汇编与受限 profiling 的观察，可以看到数据访问模式与指令依赖结构往往比表面计算量更直接地决定现代 CPU 上的实际执行时间。

关键词： CPU 架构编程，cache 局部性，超标量，循环展开，性能测试

目录

1 实验任务 **3**

2 实验环境与测试方法 **3**
2.1 实验环境 3
2.2 环境迁移与构建 4
2.3 测试数据与规模设计 4
2.4 计时方式 4

3 算法设计与分步优化 **5**
3.1 矩阵列内积 5
3.2 求和 6

4 实验结果与分析 **7**
4.1 矩阵列内积实验 7
4.2 求和实验 7
4.3 汇编观察与 profiling 8

5 结论 **9**

1 实验任务

本次实验围绕 CPU 架构中的两个典型性能问题展开：

1. 给定 $n \times n$ 矩阵 B 和向量 a，计算每一列与向量的内积

$$s_j = \sum_{i=0}^{n-1} B_{i,j} a_i.$$

重点观察矩阵访问顺序与 cache 局部性的关系。

2. 给定 n 个浮点数，计算其和

$$S = \sum_{i=0}^{n-1} x_i.$$

重点观察链式依赖、多链独立累加和归约方式对超标量执行的影响。

相较于最初在 WSL 上完成的版本，本次实验迁移到真实 Ubuntu 机器重新执行，代码也进一步整理为多文件工程，而不是把所有逻辑塞进一个源文件中。目录划分如下：

- `src/benchmark.*`：计时、warm-up、绑核与 profiling 封装；
- `src/fs_compat.hpp`：GCC 7 / GCC 12 的 `filesystem` 兼容层；
- `src/matrix_experiment.*`：矩阵列内积的三个实现；
- `src/sum_experiment.*`：求和的四个实现；
- `src/environment.*`、`src/profile.*`：环境记录与 profiling 结果导出；
- `src/main.cpp`：实验入口；
- `report/main.tex`：LaTeX 实验报告。

2 实验环境与测试方法

2.1 实验环境

本次正式结果来自当前真实 Ubuntu 机器。平台信息由程序自动写入环境记录文件中：

`results/environment.txt`

表 1 列出了其中的核心配置。

项目	配置
CPU	AMD EPYC 7453 28-Core Processor
体系结构	x86_64，双路 56 核，单核单线程
L1d Cache	32 KiB
L2 Cache	512 KiB
L3 Cache	16384 KiB
操作系统	Ubuntu 18.04.6 LTS
Kernel	Linux 4.18.0-80.7.1.el8_0.x86_64
编译器	g++ 7.5.0
编译选项	`-std=c++17 -O3 -march=native`
额外控制	`-fno-tree-vectorize -fno-tree-slp-vectorize`
实际绑核	绑定到允许掩码中的 CPU 21

表 1 实验平台配置

关闭自动向量化的目的是尽量把实验重点放在 cache 友好访问和指令级并行本身，而不是让编译器自动生成 SIMD 指令掩盖差异。由于当前机器分配给本进程的 CPU 掩码为 `21-27`，程序不再死板地绑定 CPU0，而是优先尝试 CPU0，失败后自动回退到当前允许掩码中的首个逻辑核，保证单核重复测试的稳定性。

2.2 环境迁移与构建

从 WSL 迁移到当前机器后，最直接的兼容性问题来自 GCC 版本差异：当前主机是 g++ 7.5.0，标准库中的 `<filesystem>` 仍需兼容处理。因此本实验加入了 `src/fs_compat.hpp` 和 Makefile 中的 `-lstdc++fs` 逻辑，使工程既能在旧版 GCC 7 上构建，也能继续在新版本 GCC 上运行。

本次实验的构建与执行命令如下：

Listing 1 编译、运行与导出汇编

```
make clean all
make run
make asm
```

2.3 测试数据与规模设计

为便于复现实验与正确性校验，两部分实验都采用固定公式生成数据：

$$a_i = 0.5 + ((19i + 13) \bmod 127)/127.0,$$

$$B_{i,j} = 0.2 + ((137i + 23j) \bmod 257)/257.0.$$

矩阵实验选取

$$n = \{64, 128, 256, 384, 512, 768, 1024, 1536, 2048\},$$

对应工作集大小为 $8n^2$ Byte。这里 $n = 256$ 时矩阵大小刚好为 512 KiB，与本机 L2 规模相当；$n = 1536$ 时矩阵大小约为 18 MiB，已经超过本机通过 sysfs 读取到的 16 MiB L3 条目。

求和实验选取

$$n = \{2^{10}, 2^{12}, 2^{14}, 2^{16}, 2^{18}, 2^{20}, 2^{22}, 2^{23}\},$$

数组大小为 $8n$ Byte，用于观察问题规模从 cache 友好区间逐步进入带宽受限区间时，各种实现的性能变化。

2.4 计时方式

程序使用 `std::chrono::steady_clock` 计时。对每组规模，不预设固定运行次数，而是重复执行待测函数直到累计时间超过 250 ms，再以总时间除以迭代次数得到平均耗时。这样可以明显减小小规模测试中过短运行时间带来的抖动。每次正式计时之前，还会先执行 2 轮 warm-up。

计时框架的核心思想是“先让一次测量足够长，再计算单次平均开销”，其流程与实际工程代码一致，可概括为：

Listing 2 累计时长阈值计时框架

```
for (int warmup = 0; warmup < 2; ++warmup) {
    do_not_optimize(run_once());
}

```

```
size_t iterations = 1;
while (true) {
    auto start = Clock::now();
    double checksum = 0.0;
    for (size_t i = 0; i < iterations; ++i) {
        checksum += run_once();
    }
    auto end = Clock::now();
    double total_ms = duration_ms(end - start);
    if (total_ms >= 250.0) {
        avg_ms = total_ms / iterations;
        break;
    }
    iterations *= 2;
}
```

这里将迭代次数按 2 倍递增，可以在小规模问题上快速越过计时精度不够的区间，同时避免人为为每个规模单独手调重复次数。

3 算法设计与分步优化

3.1 矩阵列内积

矩阵部分按“逐步优化”的方式实现了三个版本：

1. **平凡算法**：外层按列循环，内层按行访问 `matrix[row * n + col]`。该写法与 C/C++ 的行主存储布局不匹配，跨行访问步长为 n，空间局部性差。
2. **行优先算法**：改为外层按行扫描矩阵，内层按列把该行对所有输出元素的贡献累加到输出数组中，即 `out[col]`。这样内层变成连续访问，更符合 cache line 的利用方式。
3. **行优先 + 4 路展开**：在行优先基础上，对内层循环做 4 路手工展开，减少循环控制开销，也让处理器获得更稳定的指令流。

矩阵实验的关键不是减少乘加次数，因为三个版本的计算量同阶，真正拉开差距的是访存模式是否与行主存储布局匹配。循环展开只是第二层优化，收益建立在访问顺序已经被修正的前提之上。

对应到代码层面，矩阵实验最关键的变化并不是公式本身，而是循环次序和连续访存方式：

Listing 3 矩阵列内积的核心实现片段

```
// naive: outer loop on columns, inner loop on rows
for (size_t col = 0; col < n; ++col) {
    double acc = 0.0;
    for (size_t row = 0; row < n; ++row) {
        acc += matrix[row * n + col] * vec[row];
    }
    out[col] = acc;
}

// row-major + unroll4: scan a whole row contiguously
for (size_t row = 0; row < n; ++row) {
    const double factor = vec[row];
```

```
    const double* row_ptr = matrix + row * n;
    size_t col = 0;
    for (; col + 3 < n; col += 4) {
        out[col]     += row_ptr[col]     * factor;
        out[col + 1] += row_ptr[col + 1] * factor;
        out[col + 2] += row_ptr[col + 2] * factor;
        out[col + 3] += row_ptr[col + 3] * factor;
    }
}
```

3.2 求和

求和部分也按优化阶段实现了四个版本：

1. **链式累加**：单个累加器顺序求和，依赖链最长；
2. **两路独立累加**：把一条依赖链拆成两条；
3. **四路独立累加**：进一步拆成四条独立链；
4. **两两归约**：先把相邻元素配对相加，再不断缩减问题规模直至只剩一个结果。

这里的核心思想是减少串行依赖。链式累加中每一次加法都依赖上一次结果，而两路、四路独立累加把多条累加链分散到不同寄存器上，更容易被超标量流水线并行发射。归约法虽然理论上也缩短了依赖深度，但它需要读写中间结果，因此性能上还要同时接受额外访存成本的检验。

在实现上，超标量优化最直接的体现就是把一条长依赖链拆成多条独立链：

Listing 4 求和实验的关键实现片段

```
// naive: one dependency chain
double sum = 0.0;
for (size_t i = 0; i < n; ++i) {
    sum += data[i];
}

// superscalar4: four independent accumulators
double s0 = 0.0, s1 = 0.0, s2 = 0.0, s3 = 0.0;
size_t i = 0;
for (; i + 3 < n; i += 4) {
    s0 += data[i];
    s1 += data[i + 1];
    s2 += data[i + 2];
    s3 += data[i + 3];
}
double sum4 = (s0 + s1) + (s2 + s3);
```

相较之下，两两归约虽然也缩短了理论依赖深度，但必须不断把中间结果写回数组并重新读取，因此它更依赖具体实现的访存效率。

4 实验结果与分析

4.1 矩阵列内积实验

表 2 给出了矩阵实验结果。加速比 1 表示“平凡算法 / 行优先”，加速比 2 表示“平凡算法 / 行优先 + 4 路展开”。

n	平凡/ms	行优先/ms	行优先 +U4/ms	加速比 1	加速比 2
64	0.003036	0.002603	0.001974	1.17	1.54
128	0.021740	0.010679	0.007805	2.04	2.79
256	0.117871	0.041737	0.031798	2.82	3.71
384	0.200719	0.092629	0.070405	2.17	2.85
512	1.325131	0.165555	0.125567	8.00	10.55
768	1.544662	0.370398	0.285798	4.17	5.40
1024	7.249227	0.650013	0.501884	11.15	14.44
1536	16.459645	1.522088	1.182555	10.81	13.92
2048	65.956119	2.656099	2.085891	24.83	31.62

表 2 矩阵列内积实验结果

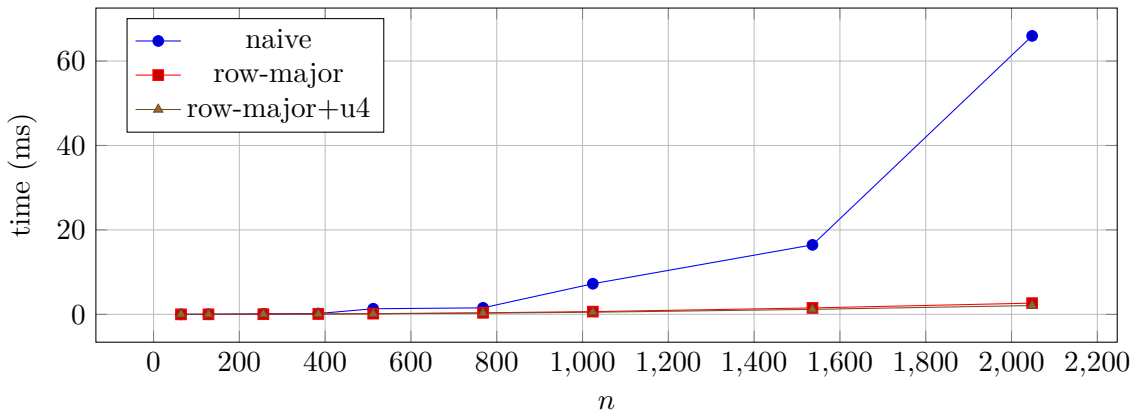

图 1 矩阵列内积三种实现的运行时间

矩阵实验有三点现象最值得注意：

1. 当 $n = 256$ 时，矩阵大小约为 512 KiB，已经与本机 L2 规模接近。此时行优先和展开版本相较于平凡算法已有 2.8 倍到 3.8 倍提升，说明单纯修正访问顺序就足以产生明显收益。
2. 当 $n = 512$ 以后，矩阵大小达到 2 MiB，已经明显越过 L2。平凡算法时间从 0.201 ms 增长到 1.325 ms，而行优先 + 4 路展开只增长到 0.126 ms，加速比迅速跃升到 10.55 倍，cache 友好访问开始主导差异。
3. 当 $n = 1536$ 和 $n = 2048$ 时，工作集进一步逼近并超过本机读取到的 16 MiB L3 条目。此时平凡算法退化尤为明显，而优化版本仍然保持顺滑增长，最终在 $n = 2048$ 时取得 31.62 倍的最高加速比。

从结果看，矩阵实验中“循环展开”的收益始终存在，但第一决定因素仍然是访问顺序是否匹配行主存储布局。只有先把访存模式从“跨行跳跃”改成“按行连续扫描”，后续的 4 路展开才真正有发挥空间。

4.2 求和实验

表 3 给出了求和实验结果。三种加速比分别以链式累加为基准。

n	链式/ms	两路/ms	四路/ms	归约/ms	两路加速比	四路加速比	归约加速比
1024	0.000879	0.000437	0.000232	0.000494	2.01	3.78	1.78
4096	0.003682	0.001826	0.000954	0.002101	2.02	3.86	1.75
16384	0.014649	0.007359	0.003827	0.009219	1.99	3.83	1.59
65536	0.059804	0.029508	0.015424	0.039763	2.03	3.88	1.50
262144	0.237273	0.118453	0.061373	0.290955	2.00	3.87	0.82
1048576	0.959793	0.474628	0.255036	1.169739	2.02	3.76	0.82
4194304	3.860151	2.043320	1.521183	5.060711	1.89	2.54	0.76
8388608	8.238081	4.055658	3.018976	11.601031	2.03	2.73	0.71

表 3 求和实验结果

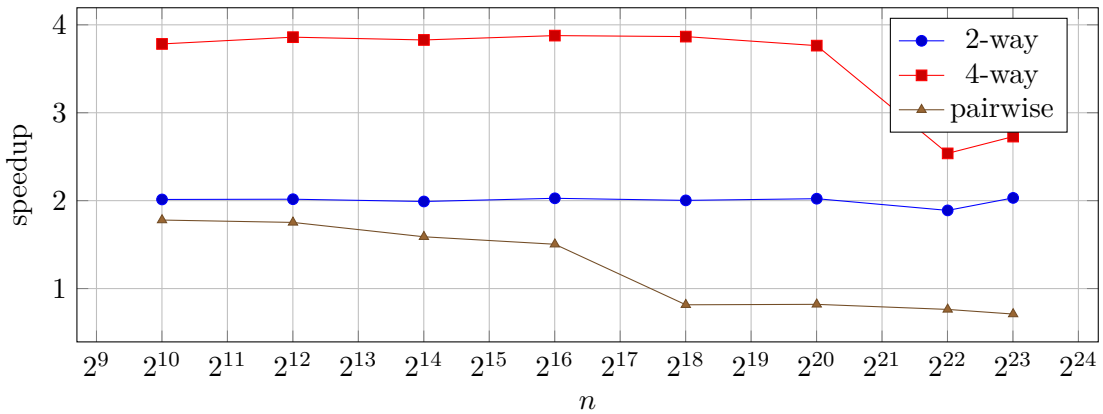

图 2 求和优化版本相对链式累加的加速比

求和实验说明了超标量优化与内存系统约束之间的关系：

1. 两路独立累加几乎在所有规模下都稳定维持约 2 倍加速，说明把单条依赖链拆成两条后，处理器基本能稳定并行调度这两组加法。
2. 四路独立累加表现最好，在 $n = 65536$ 时达到本次实验最高加速比 3.88 倍，已经非常接近理想的 4 倍。这说明在数据仍主要受 cache 支持时，多路独立累加能较充分地发挥超标量流水线能力。
3. 当规模增长到 2^{22} 和 2^{23}，数组大小分别为 32 MiB 和 64 MiB，已经明显超过 L3 量级，四路版本的加速比下降到 2.54 倍和 2.73 倍。此时瓶颈开始更多地转移到内存带宽，而不再只是加法依赖链长度。
4. 两两归约只在很小规模时表现不错，随后很快落后于多链累加；当规模达到 262144 及以上时甚至明显慢于链式累加。原因在于归约虽然减小了依赖深度，但不断读写中间结果，带来了额外访存成本。

此外，不同求和顺序会带来微小浮点舍入差异。实测最大绝对误差出现在 $n = 8388608$ 时，量级约为 4.66×10^{-9}，远小于结果本身，可以视为浮点重排带来的正常现象。

4.3 汇编观察与 profiling

对 `results/assembly_excerpt.txt` 的观察可以看到，链式累加与多链累加在目标代码层面确实呈现出不同的依赖结构：

Listing 5 链式累加与多链累加的关键汇编片段

```
sum_naive:
    vaddsd xmm0, xmm0, QWORD PTR [rdi]
```

```

sum_superscalar2:
    vaddsd xmm0, xmm0, QWORD PTR [rax]
    vaddsd xmm1, xmm1, QWORD PTR [rax+0x8]

sum_superscalar4:
    vaddsd xmm1, xmm1, QWORD PTR [rax]
    vaddsd xmm3, xmm3, QWORD PTR [rax+0x8]
    vaddsd xmm0, xmm0, QWORD PTR [rax-0x10]
    vaddsd xmm2, xmm2, QWORD PTR [rax-0x8]
```

这与时间结果是吻合的：`sum_naive` 只维护一条累加链，而两路、四路版本则显式引入多个彼此独立的累加寄存器，因而更容易被超标量流水线并行发射。

我同时保留了 `perf_event_open` 版本的 profiling 代码，并在 `results/profile_results.csv` 中记录结果。但当前环境下所有硬件事件都返回 `perf_event_open failed: No such file or directory`。结合 `results/environment.txt` 中记录的 `perf_event_paranoid=2` 可以推断，失败原因并不是简单的 paranoid 等级过高，而更像是当前内核/容器环境未向用户态暴露对应的 PMU 事件接口。因而，本报告对性能原因的分析主要依据两类证据：一是不同问题规模下的运行时间趋势，二是反汇编得到的目标代码结构。

5 结论

在真实 Ubuntu 机器上重新完成本次实验后，可以得到以下结论：

1. 对矩阵列内积问题，决定性能的首要因素是访存模式。将逐列访问改写为行优先访问后，随着工作集越过 L2/L3 边界，加速比持续放大；本次实验在 $n = 2048$ 时，行优先 + 4 路展开相对平凡算法达到 31.62 倍加速。
2. 对求和问题，多链独立累加可以显著缓解串行依赖。两路独立累加稳定接近 2 倍，四路独立累加最高达到 3.88 倍，较好地展示了超标量架构对无依赖指令流的利用能力。
3. 两两归约并非天然更快。它虽然缩短了依赖深度，但因为引入中间结果写回，在当前实现和当前平台上，当问题规模增大后反而会输给简单的多链累加。
4. 从 WSL 迁移到真实 Ubuntu 平台后，实验结果更能反映真实硬件行为；同时，工程上也需要考虑 GCC 版本兼容、CPU 可用掩码和 profiling 接口可用性等实际问题。

综合来看，本次实验较为直观地说明了现代 CPU 程序性能并不只由计算量决定。数据访问顺序、指令依赖结构、cache 层次以及系统环境对硬件计数器的支持情况，都会直接影响最终测得的执行时间。